Análisis y alcance de la solución

Prototipo de consola en Java: administración de recursos mediante herencia y polimorfismo.

Propósito

Simular la operación de módulos del satélite Quetzal-2 y sus estaciones de tierra. El operador consulta la misión y avanza ciclos; cada módulo actúa según su estado y los recursos disponibles.

ID	Requisito	Responsable	Criterio de aceptación
R1	Listar módulos	Mision / VistaConsola	Mostrar los 10 módulos y sus atributos.
R2	Buscar por ID o nombre	Mision.buscar	Informar coincidencias o ausencia de resultados.
R3	Ordenar por costo	Mision.catalogoPorCosto	Orden ascendente sin modificar el ciclo.
R4	Avanzar un ciclo	Mision / subclases de Modulo	Operación secuencial; recursos persistentes.
R5	Resumen de comunicaciones	Mision.resumenComunicaciones	Totales, capacidad activa y todos los empates.

Restricciones: una lista polimórfica principal, los tres tipos de módulos, MVC y diagrama UML 1.4. La interfaz exigida es un menú de consola. La entrega contiene fuentes Java, compilados y este análisis.

Reglas de negocio y estado inicial

Las capacidades permanecen constantes. Los acumulados cuentan operaciones efectivas.

Módulo	Condición para operar	Efecto sobre la misión
Vuelo	Activo y energía suficiente.	Consume toda la energía requerida; agrega su capacidad de MB en órbita. No admite recolección parcial.
Tierra	Activo, energía suficiente y datos.	Descarga min(capacidad, datos). Paga el consumo completo y actualiza los históricos de misión y módulo.
Energía	Activo.	Genera su capacidad fija. No se modelan baterías.

Estado de partida

Ciclo 10 · Energía: 20 unidades
Datos en órbita: 80 MB
Datos descargados: 160 MB
Histórico de vuelo: 240 MB = 80 + 160

La carga contiene 4 módulos de vuelo, 3 de tierra y 3 de energía. Incluye inactivos y un empate entre dos antenas con 60 MB descargados.

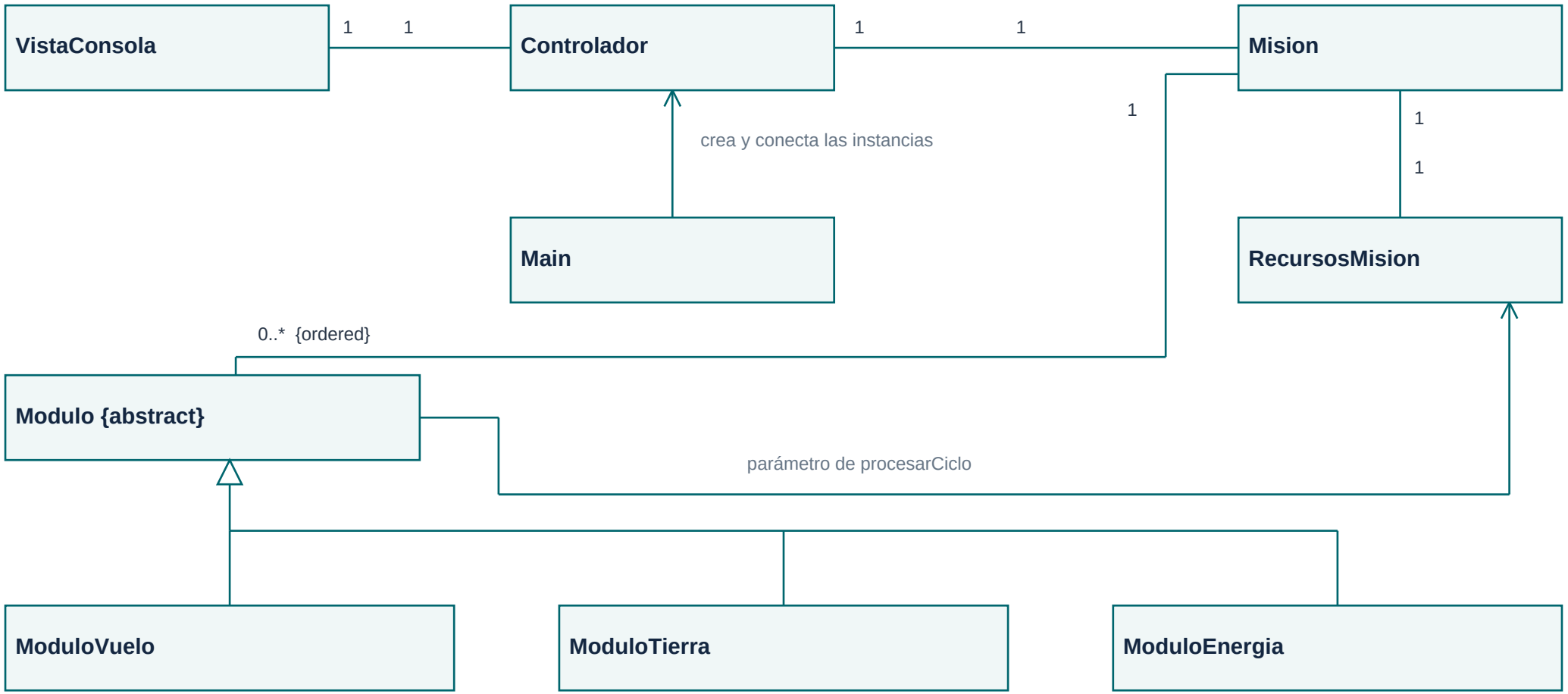
Decisiones explícitas

Salud informativa entre 0 y 100; actividad independiente. Cada antena tiene estación. Búsqueda por ID exacto o nombre completo sin distinguir mayúsculas.

Sin operación no se consumen recursos ni se incrementan ciclos efectivos. El estado se conserva entre ciclos, pero se reinicia al abrir el programa.

Arquitectura y relaciones UML

Vista general; las páginas 4 a 7 detallan atributos, constructores y operaciones declaradas.



Línea continua: asociación. Flecha discontinua: dependencia. Triángulo vacío: herencia. No se usa composición: el código recibe referencias a módulos y recursos ya creados.

Contrato base y recursos compartidos

Los miembros heredados no se repiten en las subclases. + público · - privado · # protegido.

Modulo {abstract}

- id: String {frozen}
- nombre: String {frozen}
- salud: int {frozen}
- activo: boolean
- costo: double {frozen}

Modulo(id: String, nombre: String, salud: int, activo: boolean, costo: double)
+ procesarCiclo(recursos: RecursosMision): String {abstract}
+ getId(): String
+ getNombre(): String
+ getSalud(): int
+ isActivo(): boolean
+ setActivo(activo: boolean): void
+ getCosto(): double
+ toString(): String
+ equals(otro: Object): boolean {leaf}
+ hashCode(): int {leaf}

RecursosMision

- energia: long
- datosOrbita: long
- datosDescargados: long

+ RecursosMision(energia: long, datosOrbita: long, datosDescargados: long)
+ getEnergia(): long
+ getDatosOrbita(): long
+ getDatosDescargados(): long
+ generar(cantidad: long): void
+ recolectar(consumo: long, datos: long): void
+ descargar(consumo: long, datos: long): void
+ toString(): String

{abstract}: operación o clase abstracta. {leaf}: método final. {frozen}: referencia o valor final después de construir el objeto; no implica que el objeto referenciado sea inmutable.

Instrumentos y estaciones de tierra

Constructores completos, consultas y sobrescritura de la operación de ciclo.

ModuloVuelo

- instrumento: Instrumento {frozen}
- datosPorCiclo: long {frozen}
- consumo: long {frozen}
- ciclos: long

+ ModuloVuelo(id: String, nombre: String, salud: int, activo: boolean, costo: double, instrumento: Instrumento, datosPorCiclo: long, consumo: long, ciclos: long)
+ getInstrumento(): Instrumento
+ getDatosPorCiclo(): long
+ getConsumo(): long
+ getCiclos(): long
+ getTotalRecolectado(): long
+ procesarCiclo(recursos: RecursosMision): String
+ toString(): String

ModuloTierra

- estacion: String {frozen}
- capacidad: long {frozen}
- consumo: long {frozen}
- totalDescargado: long

+ ModuloTierra(id: String, nombre: String, salud: int, activo: boolean, costo: double, estacion: String, capacidad: long, consumo: long, totalDescargado: long)
+ getEstacion(): String
+ getCapacidad(): long
+ getConsumo(): long
+ getTotalDescargado(): long
+ procesarCiclo(recursos: RecursosMision): String
+ toString(): String

Tipo anidado público ModuloVuelo.Instrumento: CAMARA, SENSOR (enum de Java). El histórico de vuelo se calcula como datosPorCiclo × ciclos; tierra almacena las descargas efectivas.

Generación y administración de la misión

Mision conserva una única ArrayList<Modulo>; las listas de consulta son copias temporales.

ModuloEnergia

- energiaPorCiclo: long {frozen}
- ciclos: long

+ ModuloEnergia(id: String, nombre: String, salud: int, activo: boolean, costo: double, energiaPorCiclo: long, ciclos: long)
+ getEnergiaPorCiclo(): long
+ getCiclos(): long
+ getTotalGenerado(): long
+ procesarCiclo(recursos: RecursosMision): String
+ toString(): String

Mision

- modulos: ArrayList<Modulo> {frozen}
- recursos: RecursosMision {frozen}
- ciclo: long

+ Mision(modulos: List<Modulo>, recursos: RecursosMision, ciclo: long)
+ getModulos(): List<Modulo>
+ getRecursos(): RecursosMision
+ getCiclo(): long
+ buscar(texto: String): List<Modulo>
+ catalogoPorCosto(): List<Modulo>
+ avanzarCiclo(): List<String>
+ resumenComunicaciones(): String

Despacho polimórfico

avanzarCiclo() recorre referencias Modulo e invoca procesarCiclo(recursos). Java selecciona la implementación de cada subclase. Los recursos se actualizan antes de pasar al siguiente módulo. El filtro instanceof del resumen se usa para consultar antenas, no para elegir la operación del ciclo.

Controlador, vista y arranque

El modelo no usa System.in ni System.out. La vista concentra la interacción por consola.

Controlador

- mision: Mision {frozen}
- vista: VistaConsola {frozen}

+ Controlador(mision: Mision, vista: VistaConsola)
+ ejecutar(): void

VistaConsola

- entrada: Scanner {frozen}

+ VistaConsola() {implícito}
+ mostrar(texto: String): void
+ leer(pregunta: String): String
+ menu(): String
+ mostrarModulos(modulos: List<Modulo>): void

Main

(sin atributos)

+ Main() {implícito}
+ crearMision(): Mision
+ main(args: String[]): void

Separación de responsabilidades

El controlador interpreta la opción y coordina llamadas. La vista captura entradas y presenta resultados. Main crea la misión y conecta componentes. Los métodos estáticos aparecen subrayados. Los constructores implícitos se indican como tales.

Validación y preparación de la entrega

Evidencia: 37 verificaciones automáticas superadas y recorrido de todas las opciones del menú.

Escenario	Resultado comprobado
Ejemplo del enunciado	Energía 17; órbita 5 MB; descargados 10 MB al terminar un ciclo.
Operaciones bloqueadas	Inactivos, falta de energía y ausencia de datos no consumen recursos.
Descarga parcial	Descarga los MB disponibles y consume la energía completa.
Orden y continuidad	El catálogo no reordena la lista; el siguiente ciclo conserva recursos.
Consultas e históricos	Búsquedas, todos los empates, capacidad activa y rechazo de IDs duplicados.
Conservación en 20 ciclos	Recolectado = órbita + descargado; energía no negativa; históricos coincidentes.

Archivos y ejecución

src: fuentes Java organizadas por responsabilidad.
bin: compilados para Java 17 o posterior.
compilar.ps1: recompilación del proyecto.
ejecutar.cmd: apertura del menú.
README.md: instrucciones y pruebas.

Fuente y límites

Enunciado: Ejercicio 5 - Polimorfismo - Temario C - Defensa de QTZ2, pp. 1-4.

La interfaz requerida es de consola. La publicación en GitHub es un paso de entrega independiente; no forma parte de las pruebas funcionales.